



FOR SOFTWARE ENGINEERS

One Request. 80 Queries.

GraphQL with Spring Boot, in depth

The client gets one elegant query. This is everything the server now has to get right.

One shop served twice, by a REST controller and a GraphQL controller over the same data. Everything on this sheet was built and measured, not assumed.

Spring Boot 4.1.1 with Spring for GraphQL 2.0.5, on Java 25 and PostgreSQL 17.

Every version on this sheet was current on 26 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

In scope

- The schema, the controller annotations and execution
- N plus one, batching and the per request cache
- Errors, status codes and field level security
- Pagination, limits, introspection and schema evolution
- Subscriptions and testing

Out of scope

- Federation, which has its own sheet
- Performance benchmarks, which were not measured

Numbers are from one run of the script. They are this application, not a law.

01 The findings, in one table

Order list screen	REST 21 requests; GraphQL 1
Nested fields, schema mapping	20, 20 and 40 selects
Same request, batch mapping	1, 1 and 1
Unmapped exception	INTERNAL_ERROR, message hidden
Broken query, two response types	200, then 400
60 aliases, depth limit only	60 selects
Same, complexity limit	rejected, 120 over 100
Field with no resolver	null, no error

Every row is reproduced by [scripts/verify.sh](#).

02 What changes from Spring MVC

A REST controller maps a URL to a method, and the server decides the shape of the response. A GraphQL controller maps methods to fields in a schema: query mapping and mutation mapping for the root fields, subscription mapping for streams, and schema mapping for any field on any type, receiving its parent.

Every request is parsed, validated against the schema, then executed field by field. Every error and every limit belongs to one of those stages.

ShopController.java: the root fields

```
@QueryMapping
public Order order(@Argument Long id) {
    return orders.findById(id).orElseThrow(() → new OrderNotFoundException(id));
}

// first/after and last/before arrive as a ScrollSubrange; the Window that
// comes back becomes an OrderConnection with edges, cursors and pageInfo.
@QueryMapping
public Window<Order> orders(ScrollSubrange subrange) {
    ScrollPosition position = subrange.position().orElse(ScrollPosition.keySet());
    int count = subrange.count().orElse(10);
    return orders.findBy(position, Limit.of(count), Sort.by("id"));
}

@MutationMapping
public Order shipOrder(@Argument Long id) {
    Order order = service.ship(id);
    statusChanges.tryEmitNext(order);
    return order;
}

@SubscriptionMapping
public Flux<Order> orderStatus(@Argument Long orderId) {
    return statusChanges.asFlux().filter(order → order.getId().equals(orderId));
}
```

application.properties

```
server.port=8201
spring.graphql.http.path=/graphql
spring.graphql.websocket.path=/graphql-ws
spring.graphql.schema.introspection.enabled=false
```

Spring checks the schema against the controllers at startup and reports anything unmapped.

Spring for GraphQL reference documentation, checked 2026-09-26



03 The schema is the contract

Object types say what can come out, input types what can go in. Fields are nullable by default: on an output, non-null is a promise; on an input, a requirement. A list has two: the list is there, and no entry is null.

schema.graphqls

```
type Query {
  order(id: ID!): Order
  orders(first: Int, after: String, last: Int, before: String): OrderConnection!
}

type Mutation {
  shipOrder(id: ID!): Order!
}

type Subscription {
  orderStatus(orderId: ID!): Order!
}

type Order {
  id: ID!
  status: OrderStatus!
  total: Int! @deprecated(reason: "Renamed to amount")
  amount: Int!
  placedAt: String!
  customer: Customer!
  lines: [OrderLine!]!
}

type OrderLine {
  quantity: Int!
  unitPrice: Int!
  product: Product!
}

type Product {
  sku: ID!
  name: String!
  description: String!
  price: Int!
}

type Customer {
  id: ID!
  name: String!
  tier: String!
  email: String
}

enum OrderStatus {
  PLACED
  PAID
  SHIPPED
}
```

04 Why GraphQL exists, measured

The order list screen needed 21 REST requests and 2,410 bytes; GraphQL answered it in one request and 1,608 bytes. The detail screen: 1,124 bytes against 135, because REST sent product descriptions the screen never shows.

This application, not a law. REST selects resources; GraphQL selects fields. Fewer round trips and smaller responses do not mean less work on the server.

REST 21 requests, 2,410 bytes; GraphQL 1 request, 1,608 bytes.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.

05 One request, eighty queries

With schema mapping, each nested field is resolved one parent at a time: 20 customer selects, 20 order line selects, 40 product selects, for one request. Batch mapping hands the method every parent that needs the field, and the method makes one batched query. Same request: one, one and one.

Batching combines keys; the data loader underneath also caches by key, for one request only. The same request again ran one more product query. A cache that outlives the request needs a staleness decision.

ShopController.java: one batched query per field

```
@BatchMapping
public Map<Order, Customer> customer(List<Order> batch) {
    Map<Long, Customer> byId = customers
        .findAllById(batch.stream().map(Order::getCustomerId).toList())
        .stream()
        .collect(Collectors.toMap(Customer::getId, Function.identity()));
    return batch.stream().collect(Collectors.toMap(
        Function.identity(), order -> byId.get(order.getCustomerId())));
}
```

scripts/verify.sh

```
batch mapping: customer 1, order_line 1, product 1
schema mapping: customer 20, order_line 20, product 40
```

Schema mapping: 20, 20 and 40 selects. Batch mapping: 1, 1 and 1.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.

One product select for a request; one more for the same request again.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.

06 Errors are data

An exception nobody maps reaches the client as `INTERNAL_ERROR` with its message hidden. An exception resolver turns a missing order into `NOT_FOUND` with its message, and the response carries data and errors together.

ShopExceptionResolver.java

```
@Component
public class ShopExceptionResolver extends DataFetcherExceptionHandlerAdapter {

    @Override
    protected GraphQLError resolveToSingleError(Throwable ex, DataFetchingEnvironment env) {
        if (ex instanceof OrderNotFoundException) {
            return GraphQLErrorBuilder.newError(env)
                .errorType(ErrorType.NOT_FOUND)
                .message(ex.getMessage())
                .build();
        }
        return null;
    }
}
```

Without a resolver: `INTERNAL_ERROR`. With one: `NOT_FOUND`.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.

07 Security below the URL

Every request goes to one URL, so securing the URL only decides who reaches the endpoint. Pre authorize on the method that resolves one field: anonymous got `UNAUTHORIZED`, a clerk `FORBIDDEN`, the admin the email, and every caller got the name. In a real application, keep the rule on the service method that owns the data.

ShopController.java: one field, one rule

```
@PreAuthorize("hasRole('ADMIN')")
public String email(Customer customer) {
    return customer.getEmail();
}
```

Anonymous `UNAUTHORIZED`, clerk `FORBIDDEN`, admin sees the email.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.

08 HTTP is transport

The same broken queries returned 200 when the client asked for application/json, and 400 when it asked for application/graphql-response+json, the media type from the GraphQL over HTTP specification. A missing order was 200 either way: the request was valid and failed during execution. GET on this Spring GraphQL 2.0 endpoint returned 405.

```
scripts/verify.sh
```

```
application/json: syntax error 200 , unknown field 200 , missing order 200
application/graphql-response+json: syntax error 400 , unknown field 400 , missing order 200
GET /graphql?query=...: 405
```

200 under application/json, 400 under application/graphql-response+json; missing order 200 either way.

Run on a real machine, 2026-09-26. Reproduce it with scripts/verify.sh in the course repository.

09 Cursor pagination

First and after arrive as a scroll subrange; a Spring Data window comes back and Spring turns it into a connection with edges, cursors and page info. Page two was 4, 5, 6. In this implementation the opaque cursor encodes a key set position. Cursors trade random access for stability.

```
ShopController.java and Repositories.java
```

```
@QueryMapping
public Order order(@Argument Long id) {
    return orders.findById(id).orElseThrow(() -> new OrderNotFoundException(id));
}

// first/after and last/before arrive as a ScrollSubrange; the Window that
// comes back becomes an OrderConnection with edges, cursors and pageInfo.
@QueryMapping
public Window<Order> orders(ScrollSubrange subrange) {
    ScrollPosition position = subrange.position().orElse(ScrollPosition.keySet());
    int count = subrange.count().orElse(10);
    return orders.findBy(position, Limit.of(count), Sort.by("id"));
}

Window<Order> findBy(ScrollPosition position, Limit limit, Sort sort);
```

The cursor decoded to a key set position on id.

Run on a real machine, 2026-09-26. Reproduce it with scripts/verify.sh in the course repository.

10 Flexibility is attack surface

Sixty aliases at depth two passed a depth limit and ran 60 queries. A complexity limit scores every field and rejected the same query at 120 over 100 before any query ran. Depth stops deep queries; complexity stops wide ones. Weight lists by their size, cap page sizes, and add timeouts.

Introspection is on by default and off with one property. That hides the discovery mechanism and secures nothing else.

```
GraphQLLimits.java

@Configuration
public class GraphQLLimits {

    @Bean
    public MaxQueryDepthInstrumentation maxDepth() {
        return new MaxQueryDepthInstrumentation(6);
    }

    @Bean
    public MaxQueryComplexityInstrumentation maxComplexity() {
        return new MaxQueryComplexityInstrumentation(100);
    }
}
```

Depth limit only: 60 selects. Complexity: rejected, 120 over 100.

Run on a real machine, 2026-09-26. Reproduce it with scripts/verify.sh in the course repository.

11 A field that fails in silence

A field added to the schema with no code to resolve it returns null with no error. Spring's schema inspection report named it at startup. Read that report, or enforce it in the build.

```
Startup log: schema inspection

Unmapped fields: {Order=[trackingNumber]}
```

Unmapped fields: {Order=[trackingNumber]}

Run on a real machine, 2026-09-26. Reproduce it with scripts/verify.sh in the course repository.

12 Changing a schema

Add the new field, deprecate the old one with a reason, and watch which operations still ask for it. Deprecated, total still answered. Removed, the old query failed with a 400 validation error.

Adding a nullable field or an optional argument is usually safe. Removing or renaming a field, making an input required, or making an output nullable is breaking.

```
schema.graphqls: rename without breaking
```

```
total: Int! @deprecated(reason: "Renamed to amount")
amount: Int!
```

Removed: 400, ValidationError, field total undefined.

Run on a real machine, 2026-09-26. Reproduce it with scripts/verify.sh in the course repository.

13 Mutations, transactions and production

Top level mutation fields run in order. Put the transaction on the service use case, exactly as behind REST, never around a whole client chosen query. A parsed document cache, a persisted operation and a safelist are three different things. With every path at /graphql, observe per operation and per field, and never use raw query text as a metric label. File uploads are not part of core GraphQL: upload over HTTP and pass a reference.

```
OrderService.java: the transaction lives here
```

```
@Transactional
public Order ship(Long id) {
    Order order = orders.findById(id).orElseThrow(() -> new OrderNotFoundException(id));
    order.ship();
    return order;
}
```

Spring for GraphQL does not directly support the multipart request convention.

Spring for GraphQL reference documentation, checked 2026-09-26

14 Testing in layers

A GraphQL test slice starts the schema, controllers and exception resolver with no database and no server. It proves the contract and the wiring, not the SQL and not the status codes. Add integration tests against a real database and transport tests over HTTP or WebSocket.

```
ShopControllerTests.java

// A slice: the schema, the controller and the exception resolver, no database.
@GraphQLTest(ShopController.class)
@Import(ShopExceptionResolver.class)
class ShopControllerTests {

    @Autowired
    GraphQLTester graphQLTester;

    @MockitoBean OrderRepository orders;
    @MockitoBean OrderLineRepository lines;
    @MockitoBean CustomerRepository customers;
    @MockitoBean ProductRepository products;
    @MockitoBean OrderService service;

    @Test
    void returnsTheFieldsAskedFor() {
        given(orders.findById(1L)).willReturn(
            Optional.of(new Order(1L, 3_000, Instant.parse("2026-09-01T10:00:00Z"))));

        graphQLTester.document("{ order(id: 1) { status total } }")
            .execute()
            .path("order.status").entity(String.class).isEqualTo("PLACED")
            .path("order.total").entity(Integer.class).isEqualTo(3000);
    }

    @Test
    void aMissingOrderIsNotFound() {
        given(orders.findById(999L)).willReturn(Optional.empty());

        graphQLTester.document("{ order(id: 999) { id } }")
            .execute()
            .errors()
            .expect(error -> error.getErrorType() == ErrorType.NOT_FOUND);
    }
}
```

Two slice tests pass with no database.

Run on a real machine, 2026-09-26. Reproduce it with `scripts/verify.sh` in the course repository.



15 The interview questions this answers

One request ran eighty queries: why, and what is the Spring fix?

Resolver or data loader?

Does a data loader cache, and why only per request?

A non-null field resolves to null: then what?

Parse, validation or execution error: which is which?

Can a response carry data and errors together?

Does GraphQL always return 200?

Offset or cursor, and why is the cursor opaque?

Depth limit or complexity limit?

Persisted query, parse cache or safelist?

Can GraphQL upload files?

How do you monitor when every path is /graphql?

How do you change a schema safely?

Where do transactions and authorization live?

When would you choose REST instead?

Before a GraphQL API goes near production

1. Every nested field that repeats is batched, and you counted the queries
2. Exceptions map to error types the client can act on
3. Sensitive fields have their own rule, ideally on the service
4. Clients read both the status code and the errors list
5. Complexity limits, page size caps and timeouts exist
6. The startup schema inspection report is read, or enforced
7. Fields are deprecated before they are removed
8. scripts/verify.sh passes on a fresh clone



Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  GitHub
https://github.com/code-with-sam-dev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  x
https://x.com/CodeWithSamDev | The course repository
https://github.com/code-with-sam-dev/spring-graphql-in-depth |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Spring for GraphQL reference
https://docs.spring.io/spring-graphql/reference/ |
|  Instagram
https://www.instagram.com/codewithsamdev | GraphQL over HTTP specification
https://graphql.github.io/graphql-over-http/draft/ |
|  Facebook
https://www.facebook.com/codewithsamdev | |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. GraphQL is a trademark of the GraphQL Foundation. PostgreSQL is a trademark of the PostgreSQL Community Association. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-26 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.